UE24CS352A Machine Learning – Mini-Project

Predicting Airbnb Listing Price Across Cities (NYC, Paris, Berlin)

https://github.com/VismayCD/airbnb-price-prediction

Vismay Chandra Dev (PES2UG24CS597)
Vishwas Hosamath (PES2UG24CS596)
Section: 5J

1. Problem statement

Nightly Airbnb prices are set by hosts by feel, which is hard for new hosts and opaque for guests. We predict the listing price from listing attributes in three cities and ask two questions: (i) which regression methods and feature groups work best, and (ii) does a model trained on some cities transfer to an unseen one? The task follows Luo, Zhou and Zhou (2019); we re-implemented it independently and consulted the authors' notebooks for reference.

2. Dataset

We use the detailed *Inside Airbnb* listing tables for New York City, Paris and Berlin (snapshot 23-06-2026; 90 columns). Raw rows: NYC 30,259, Paris 77,679, Berlin 12,776. After removing listings with a missing or zero price and those above $500/night, **19,148 (NYC), 42,089 (Paris) and 8,259 (Berlin)** remain (the label-transformation experiment keeps the uncapped data). Each city is split 7:2:1 into train/validation/test (seed 42); Berlin is used only as an unseen test city. Our snapshot is newer than the paper's and lacks *security_deposit, cleaning_fee, guests_included, extra_people, summary, transit, house_rules* and several other text columns; *instant_bookable, host_response_time, host_verifications, host_since* and *neighborhood_overview* are present but empty. Absolute scores are therefore not directly comparable

with the paper.

3. Approach

Features (all statistics fit on the training split only): **C** continuous – 21 columns after dropping absent and highly correlated ones ($|r| > 0.9$), mean-imputed, standardised; **I** 2-degree interaction terms; **O** one-hot categoricals plus multi-hot amenities (top 50), 339 columns for NYC; **T** TF-IDF (uni+bigrams) of *name, description, host_about*, each reduced to 50 dimensions by truncated SVD; **D** date features (days since the earliest date). **Label:** raw price, square root or log; metrics are computed in the transformed space. **Models:** Ridge, KNN (k=5), Random Forest (100 trees), XGBoost (300 trees, depth 6), and two neural networks: an SGD+Nesterov MLP (64-32, lr 0.005, L2 0.005, early stopping) and an Adam MLP (128-512-64, lr 1e-4, weight decay 1e-5). **Metrics:** R^2, MSE, plus Spearman correlation, mean bias, and R^2 after calibrating the price level on 100 labelled target-city listings (to separate ranking quality from price-level shift).

Transfer experiments. A neural network (C+O features, log label) is trained on NYC, Paris, or NYC+Paris and tested on Berlin under three preprocessing variants: **paper** (standardise with training statistics), **robust** (log1p on heavy-tailed counts and clip standardised values to ±5σ) and **robust+relgeo** (additionally replace latitude/longitude by km north/east of the city's median listing and distance to it).

4. Implementation overview

A single Python module (*airbnb_price.py*) implements *load_city* (price parsing, cleaning, cap) → *split_city* → *FeatureBuilder* (fit/transform; missing columns become NaN so cities with different schemas can share one pipeline) → *make_model* → *run_experiment*, with one function per paper table; *check_data.py* audits which paper features a CSV provides. Outputs (CSV tables and figures) go to *results/*. Compared with the authors' notebooks we fit TF-IDF/SVD/encoders on the training split only (they fit on all rows), added the Adam network to mirror their PyTorch model, and added the robust/relative-coordinate variants and diagnostics.

5. Results

Table 1 – NYC, continuous features, price ≤ $500, raw price.

Model	Ridge	KNN	Random Forest	NN (SGD 64-32)	NN (Adam 128-512-64)	XGBoost
Train R^2	0.423	0.731	0.960	0.723	0.743	0.892
Test R^2	**0.446**	**0.607**	**0.717**	**0.666**	**0.686**	**0.740**
Test MSE	7199	5109	3685	4344	4087	3379

Table 2 – XGBoost test R^2 by label transform and feature group.

NYC, uncapped, C features	raw	sqrt	log	log + interactions
Test R^2	0.470	0.689	0.808	0.803

NYC, capped	D only	O only	T only	C+O+T (raw)	C+O+T (log)	Paris C+O+T (raw / log)
Test R^2	0.005	0.498	0.442	0.766	0.825	0.734 / 0.826

Table 3 – Neural-network transfer to Berlin (in-city = test split of the training cities; calibrated = 100 Berlin labels).

Train on	Variant	In-city R^2	Berlin R^2	Berlin Spearman	Berlin R^2 calibrated
NYC	paper	0.792	−7922.7	0.801	0.613
Paris	paper	0.763	−2317.5	0.559	0.291
NYC+Paris	paper	0.781	0.812	0.872	0.827
NYC	robust	0.793	0.604	0.790	0.716
Paris	robust	0.789	−1.814	0.838	0.710
NYC+Paris	robust	0.785	0.822	0.877	0.843
NYC	robust+relgeo	0.802	0.058	0.854	0.801
Paris	robust+relgeo	0.794	0.752	0.839	0.771
NYC+Paris	robust+relgeo	0.791	0.481	0.866	0.816

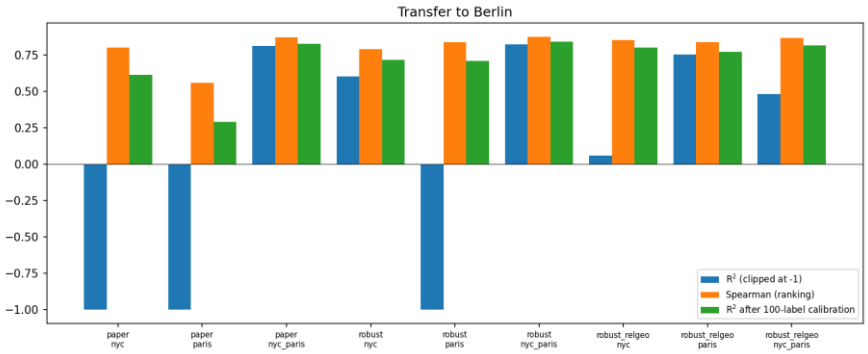


Figure 1 – Berlin transfer: raw R^2 (clipped at −1), Spearman, and R^2 after 100-label calibration.

6. Conclusions

- **Model ranking matches the paper:** XGBoost (test R^2 0.740) > Random Forest (0.717) > neural networks (0.666 / 0.686) > KNN (0.607) > Ridge (0.446). Forests overfit most (train 0.960 vs test 0.717).
- **The log label is the biggest single gain** (0.470 to 0.808 without a price cap); interaction terms gave no gain (0.803) and overfit. Categorical and text features help (all groups, log label: 0.825); date features carry no signal (0.005).
- **Naive transfer fails numerically, not because the ranking is lost.** With training-city standardisation, Berlin coordinates and extreme *maximum_nights* values reach thousands of standard deviations and the network diverges (R^2 = −7923). Robust preprocessing removes the blow-up.
- **Ranking transfers, price level does not.** Berlin Spearman is 0.79–0.88 for the non-diverged variants, close to in-city values, while the bias reflects city price levels (NYC-trained relative-coordinate model over-predicts Berlin by about 0.80 log units, roughly 2.2×). Calibrating with only 100 Berlin listings lifts R^2 to 0.71–0.84; for NYC to Berlin with relative coordinates it reaches 0.801, matching NYC's in-city 0.802. Combined NYC+Paris training with raw coordinates scores highest uncalibrated (0.822), which we suspect (not proven) is because raw coordinates let the network infer a city-level price level.
- **Limitations:** single random seed per run, a newer snapshot with fewer features than the paper, a $500 price cap, and one random 100-listing calibration draw. Prices are as provided in the files (currency not verified).